
Convincing Biology with Language: Offline Learning of Prompt-Conditioned Biological Interventions

Anonymous Author(s)

Affiliation

Address

email

Abstract

Artificial intelligence has transformed how people interact with complex technical systems across domains – from writing code to designing molecules – enabling non-experts to accomplish sophisticated tasks through natural language prompts. We envision an analogous future for biotechnology: scientists and practitioners directing living matter by describing what they want, with learned models translating those descriptions into the interventions that convince cells, organoids, or engineered biological systems to behave accordingly. Learning this mapping requires paired language-intervention-outcome data, which is expensive to collect at scale. Recent work has demonstrated language-conditioned policies for simulated cellular control, but these rely on hand-engineered reward functions and task-specific architectures, and do not exploit existing observational archives, requiring fresh experiments for every new objective. Here we demonstrate, for the first time, that a policy network can be trained offline, from a fixed archive of intervention-outcome pairs, to translate natural language prompts into interventions shown to reliably elicit desired behaviors in *in vitro* motile organoids (Xenobots). We frame this as offline contextual-bandit learning, with a three-stage pipeline: (1) a prompt-to-intervention (P2I) network maps language embeddings to intervention parameters, (2) a video archive lookup retrieves the historical Xenobot response to that intervention, and (3) a pretrained vision-language model provides a zero-shot reward signal scoring how well the observed behavior matches the original prompt. Optimizing the P2I network end-to-end with CMA-ES, and resolving objective interference between opposing behavioral objectives via a multi-head architecture, we demonstrate generalization to 40 unseen test prompts and 44 previously unobserved Xenobot videos, with 6 of 8 phenotypes exceeding the random-retrieval baseline. This work shows that language-guided biological control is learnable from purely observational data, without online experiments or hand-crafted rewards, and constitutes an intermediate step toward convincing biology with natural language.

1 Introduction

Controlling living systems is a long-standing goal across biology, medicine, and synthetic biology, yet the interface remains fundamentally low-level: practitioners specify interventions parameter-by-parameter, through trial-and-error informed by domain expertise. In other technical domains, natural language has emerged as a high-level interface to complex systems: users now specify goals — generate this code, design this molecule, plan this trajectory — and learned models translate intent into low-level action. This raises a question central to applied ML: can a similar interface be learned for biological systems, mapping natural-language descriptions of desired behavior to physical interventions that elicit it?

Two recent threads make this question tractable. Language-conditioned policies have been shown to control simulated cellular systems [11, 10], and targeted physical interventions — electrical, optogenetic, chemical — have been shown to elicit reproducible behaviors in real cells and tissues [26, 12]. However, extending this to real organisms under open-ended language faces two coupled barriers. First, collecting paired (prompt, intervention, outcome) data online is prohibitively expensive: each datum is a wet-lab experiment. Second, the objective space is inherently multi-task and contains opposing goals — *e.g.*, “stop” versus “accelerate” — which create objective interference when optimized through a shared output [25, 5]. Existing approaches sidestep both barriers by restricting to hand-crafted reward functions or single-task architectures, and as a result do not generalize to open-ended natural language.

We address both barriers by reframing language-to-intervention as an offline learning problem. Existing biological archives already contain large numbers of intervention-outcome pairs collected for purposes other than language-conditioned learning; we treat these as a fixed dataset over which a policy can be trained without any new experiments. To supply a reward signal in the absence of labeled prompt-outcome alignment, we use a pretrained vision-language model (VLM) as a zero-shot scorer [19], evaluating how well an archived outcome matches a natural-language description. To address the second barrier, the policy uses a shared backbone with one output head per behavioral phenotype, enabling independent optimization across opposing objectives while preserving shared semantic representation [20].

Our contributions are: (1) A demonstration that the language-to-intervention mapping can be learned offline, from a fixed archive of intervention-outcome pairs collected without language annotation, eliminating the need for new wet-lab experiments per objective; (2) An end-to-end pipeline combining a prompt-to-intervention policy network, archive lookup, and VLM-based zero-shot rewards, optimized with CMA-ES; and (3) A multi-head policy architecture that resolves objective interference between opposing behavioral objectives, enabling a single network to fit antagonistic prompts (*e.g.* “stop” and “accelerate”) simultaneously.

2 Related Work

Large language models have demonstrated remarkable ability to serve as general-purpose interfaces to complex technical domains. Scaling has enabled few-shot task learning [2], with subsequent work showing LLMs can generate functional code [4], design molecules [8], and assist in scientific discovery [27]. The common pattern is that users specify goals through natural language rather than low-level parameters. However, applying LLMs to biological control has proven difficult. Most prior work in this direction restricts to simulation, where prompts map to cell parameters with deterministic outcomes [11, 10]. Direct application to living organisms is rare, as the mapping from language to biological intervention lacks formal grounding — unlike code, biology offers no closed-form semantics for what move faster should mean as an electrical stimulus. Moreover, collecting paired data (prompt, intervention, outcome) requires expensive real-time experiments.

Synthetic biology has produced sophisticated engineered systems with autonomous behavior [9, 1]. Xenobots, living robots assembled from *Xenopus laevis* cells, exhibit spontaneous coordinated motion and adaptive morphology [9]. Bioelectric networks encode high-level anatomical information, suggesting goal-directed control mechanisms [12]. Yet control of these systems remains manual and low-level: practitioners tune stimulation parameters (duration, current, frequency) through trial-and-error, without any high-level behavioral interface, and the intervention-to-behavior mapping is non-linear and poorly characterized. Targeted physical interventions are nonetheless effective at the parameter level: dynamic electric cues can steer cell migration with precision [26], and optogenetic stimulation can probe biological decision-making [6]. What is missing is a learned interface between high-level behavioral specifications and these low-level intervention protocols.

Bridging language and biological behavior requires two innovations. Vision-language models trained on joint image-text embeddings [17, 14] can be used directly as zero-shot reward models, scoring the alignment between an observation and a language description [19, 11]. This approach has been explored in robotics [15, 22]; biological applications remain limited because the relevant behaviors are subtle, the visual backgrounds cluttered, and the outcomes stochastic. Learning a single policy across opposing behavioral goals (“stop” vs. “accelerate”) is a multi-task problem, and shared outputs

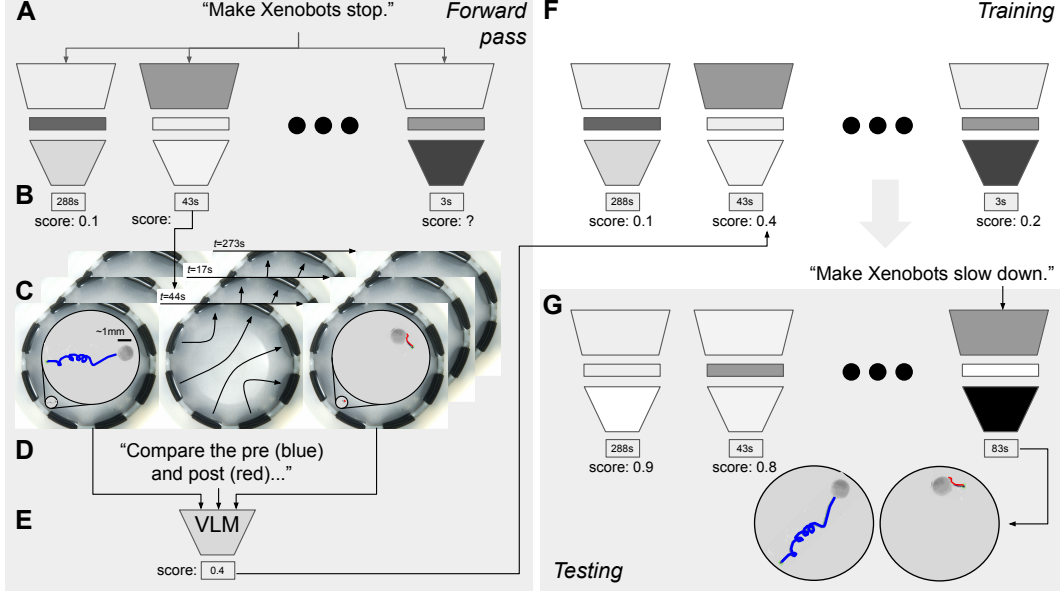


Figure 1: The pipeline. (A) User prompt → (B) P2I network → intervention duration → (C) archive lookup → (D) pre/post motion heatmaps → (E) VLM score. (F) CMA-ES evolves the P2I population on training prompts; (G) the trained P2I generalizes to held-out prompts and batches. Detailed in Sect. 3.

are known to suffer objective interference [25, 5]; task-specific heads over a shared backbone are a standard architectural remedy [23, 20].

Finally, training from a fixed archive of intervention-outcome pairs places this work in the offline RL setting [13], and specifically in the contextual-bandit regime: there are no state transitions, only terminal rewards. This regime is well-suited to derivative-free black-box optimization, for which CMA-ES [7] is a standard choice — gradient-free, robust to multi-modal landscapes, and effective in the parameter dimensionalities considered here.

3 Methods

Our approach (Fig. 1) has two phases. In the forward pass, a natural-language prompt (Sect. 3.1) is mapped by a policy to an intervention (Sect. 3.2); the closest intervention is retrieved from an archive (Sect. 3.3), which has been split into training and test interventions (Sect. 3.4); the resulting video (Sect. 3.5) and prompt are scored by a VLM (Sect. 3.6). In the backward pass, evolutionary search optimizes the policy to maximize VLM scores on training prompts (Sect. 3.7).

3.1 Prompts.

Users specify desired behavior through natural language descriptions (e.g., “make Xenobots stop moving”, “increase activity”). Each prompt is embedded using a pretrained sentence transformer (SentenceBERT; [18]), producing a 384-dimensional embedding that captures semantic meaning. This embedding serves as the input to the P2I network.

3.2 Prompt-to-Intervention (P2I):

The policy network π_{Θ} maps prompt embeddings to behavior-conditioned intervention durations. Given prompt embedding $\mathbf{e} = \phi(p) \in \mathbb{R}^{384}$ (SentenceBERT), a shared convolutional backbone extracts features (detailed architecture and parameter counts in Appendix A):

$$\mathbf{f} = \text{GlobalAvgPool}(\text{Conv1D}_{384 \rightarrow 16}(\mathbf{e})) \quad (1)$$

where the three-layer CNN reduces dimensionality ($384 \rightarrow 64 \rightarrow 32 \rightarrow 16$) with ReLU activations and dropout (0.2).

115 We compare two output designs. The *single-head baseline* produces one scalar for all behaviors:

$$\mu_{\text{single}} = \sigma(\text{MLP}_{16 \rightarrow 1}(\mathbf{f}; \Theta_{\text{single}})) \quad (2)$$

116 Under this design, prompts that demand opposing interventions – “stop” (short stimulus) versus
 117 “accelerate” (long stimulus) – backpropagate conflicting signals through the same output, so the
 118 network is pulled toward incompatible targets. Our *multi-head design* decouples this by giving each
 119 of the 8 behaviors its own output head:

$$\mu_b(\mathbf{e}) = \sigma(\text{MLP}_{16 \rightarrow 32 \rightarrow 1}(\mathbf{f}; \Theta_b)), \quad b \in \{0, 1, \dots, 7\} \quad (3)$$

120 Independent heads allow specialization (each behavior optimizes its Θ_b independently) while the
 121 shared backbone (hard parameter sharing [20]) encodes semantic features useful across all behaviors.
 122 With a 7,984-parameter backbone and 577 parameters per behavior head, the 8-behavior multi-head
 123 architecture uses 12,600 total parameters versus 8,561 for single-head, a 47.2% increase that enables
 124 behavior specialization. During training, prompt-behavior pairs are routed to the correct head via
 125 oracle labels; at test time, routing uses cosine similarity to the nearest training-prompt embedding,
 126 enabling generalization to unseen phrasings.

127 3.3 The Archive.

128 Our dataset comprises 143 Xenobot intervention videos collected from an academic institution’s
 129 bioelectricity research center (between October 2025 and January 2026. Videos available upon
 130 request with institutional approval). Each video captures pre- and post-intervention behavior at 30
 131 fps, 1280×960 resolution. Xenobot responses depend on multiple intervention parameters (current
 132 magnitude, stimulation angle, frequency, duration); we fix current, angle, and frequency, and treat
 133 *stimulus duration* $d \in [1, 327]$ seconds (median: 131s) as the single intervention variable. Each batch
 134 consists of (1) raw video, (2) a command log with exact intervention parameters and timing, and
 135 (3) trajectory files. Restricting variation to a single parameter isolates the language-to-intervention
 136 mapping from confounds in the underlying biology.

137 3.4 Data Splitting

138 The 143-batch archive is stratified into 99 training and 44 test batches, balancing difficulty tiers within
 139 each split. This stratification exploits a natural property of the data: VLM baseline scores (computed
 140 on random batch selection) reveal a bimodal distribution. *Easy-tier* behaviors (stop, motion_reduction,
 141 elimination, confinement, response_magnitude) receive consistently high baseline scores averaging
 142 0.74, indicating the archive contains abundant examples of these phenotypes. *Hard-tier* behaviors
 143 (directedness, vigor, motion_increase) receive sparse, low baseline scores averaging 0.16, indicating
 144 these phenotypes are rare in the archive (Table 1). This distribution justifies the stratified train-test
 145 split and defines the difficulty landscape: behaviors that are naturally abundant require minimal
 146 prompting, while rare behaviors demand genuine language-conditioned learning to discover them.

147 We use 80 total prompts: 40 for training and 40 held-out for testing. The 40 training prompts (8
 148 behaviors × 5 prompts) are partitioned 3:2 per behavior into optimization and validation prompts.
 149 The 99 training batches support both optimization and validation; the 44 test batches remain held-out.

150 3.5 Videos.

151 Raw Xenobot videos (30 fps, 1280×960) contain optical noise from bubbles and electrode reflections.
 152 To extract clean behavioral signals, we denoise in three steps: (1) frame differencing to localize
 153 motion regions, (2) HSV saturation thresholding ($s > 20$) to reject low-saturation artifacts (bubbles,
 154 reflections), preserving pigmented Xenobot regions, and (3) morphological cleanup of the resulting
 155 mask (detailed in Appendix D).

156 Dense optical flow (Farneback method) computed over the denoised mask yields per-frame motion
 157 magnitude. We accumulate motion magnitude separately over pre-intervention and post-intervention
 158 periods, producing two motion heatmaps that visualize behavior change:

$$\Delta H = \sum_t \text{Magnitude}_{\text{masked}}^{\text{post}}(t) - \sum_t \text{Magnitude}_{\text{masked}}^{\text{pre}}(t) \quad (4)$$

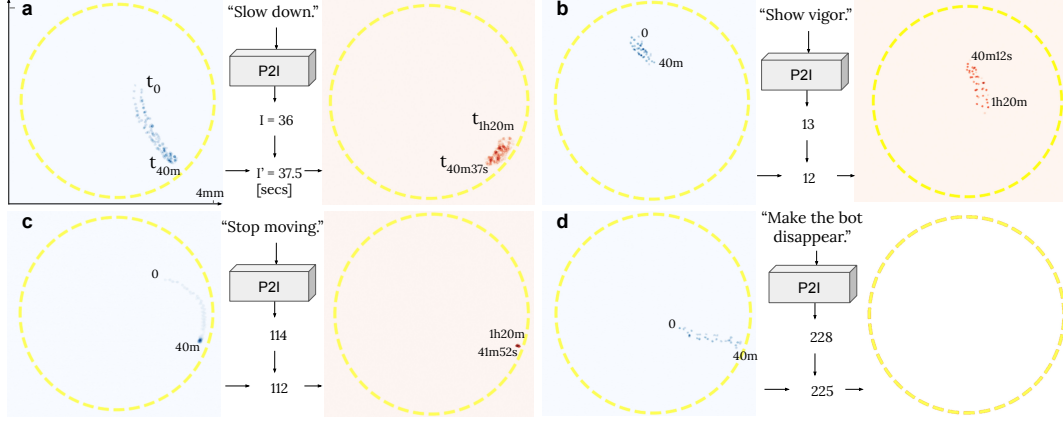


Figure 2: **Four examples of language-guided behavior produced by the trained P2I network.** For each prompt, the network outputs an intervention duration i , rounded to the nearest archived duration i' ; panels show the pre- (blue) and post-intervention (red) Xenobot trajectories in the matched archive batch. Yellow dashed circles denote the chamber boundary (4 mm diameter; 142 pixels/mm).

Figure 2 shows examples of pre- and post-intervention motion heatmaps computed from four distinct batches in the archive. These heatmaps serve as visual inputs to the VLM reward signal (Sect. 3.6), which evaluates alignment between the observed behavioral change and the command prompt given to the P2I network.

3.6 The VLM.

For each (video, prompt) pair, we compute an alignment score using a frozen vision-language model (Qwen3.5-397B served via Ollama Cloud). The VLM takes as input: (1) pre- and post-intervention motion heatmaps from Eq. 4, and (2) a query template populated with the command prompt given to the P2I network: “Compare pre (blue) vs post (red) heatmaps. How well does post-intervention behavior match: {prompt}? Score 0 - 1.” The VLM’s scalar output $s \in [0, 1]$ represents zero-shot alignment - how faithfully the observed behavior change matches the requested prompt - and serves as the reward signal. Because this score is deterministic per (video, prompt, behavior) triple under greedy decoding, we precompute the full score tensor $\mathbf{S} \in \mathbb{R}^{143 \times 80 \times 8}$ once, prior to optimization, eliminating redundant VLM calls during training. Detailed VLM query templates are provided in Appendix A.

3.7 Training

We optimize policy parameters θ to maximize average reward across all behaviors and their training prompts:

$$\max_{\theta} \frac{1}{8} \sum_{b=1}^8 \mathbb{E}_{\mathbf{e} \in P_b} [r(\theta, \mathbf{e}, b)] \quad (5)$$

where $b \in \{1, \dots, 8\}$ is the behavior index, P_b is the set of training prompts for behavior b , the action $a \in [0, 1]$ is the intervention duration produced by the policy on prompt embedding $\mathbf{e} \in \mathbb{R}^{384}$, and the reward $r \in [0, 1]$ is the VLM score (Sect. 3.6). Each behavior’s reward is averaged across its training prompts. This is a one-step contextual bandit: terminal rewards only, no state transitions, 143 samples per behavior [16, 21]. We adopt CMA-ES [7], a derivative-free algorithm suited to sparse, non-differentiable rewards, with population size $\lambda = 4 + \lfloor 3 \ln(n) \rfloor = 32$ for $n = 12,600$ multi-head parameters, run for 100 generations with initial mutation strength $\sigma_0 = 0.3$.

Table 1: The eight behavioral phenotypes used in this work. Operational definitions characterize the post-intervention motion pattern that defines each behavior; example prompts illustrate the natural-language descriptions used at training and test time. Train/Test baselines report mean VLM scores under random archive matching; these represent the inherent average behavior score of the dataset, achieved by randomly sampling durations.

Behavior	Example prompt	Train	Test
<i>Decrease in motion</i>			
stop	“Stop moving.”	0.750	0.743
motion_reduction	“Move slower.”	0.763	0.810
<i>Increase in motion</i>			
motion_increase	“Move faster.”	0.140	0.135
vigor	“Show vigor.”	0.156	0.152
response_magnitude	“Dramatic behavior change.”	0.762	0.824
<i>Structure of motion</i>			
directedness	“Show purposeful movement.”	0.150	0.190
<i>Spatial extent</i>			
confinement	“Stay confined to center.”	0.631	0.654
elimination	“Bot ceases all activity.”	0.700	0.758

184 4 Results

185 4.1 Training

186 We compared CMA-ES variants (Table 2): single-head and multi-head architectures, against random
 187 archive sampling as a reference control. The random baseline samples uniformly random durations
 188 from the archive for each prompt; over many samples, this converges to the dataset’s inherent average
 189 behavior score (0.510 across all behaviors).

190 CMA-ES multi-head achieved 0.9269 ± 0.0096 training fitness and 0.7522 ± 0.0137 validation fitness,
 191 demonstrating strong learning and generalization. Single-head CMA-ES reached 0.6664 ± 0.0218
 192 training fitness but only 0.3999 ± 0.1124 validation fitness, indicating severe overfitting: the shared
 193 backbone struggles to balance opposing objectives, causing poor transfer to held-out data. Multi-head
 194 specialization enables strong training and generalization despite the additional parameters.

Table 2: Overall training and validation fitness across optimization methods (mean $\pm$ SD across 30 seeds). Baseline is the mean VLM score from the training dataset. Values in parentheses show improvement (+) or degradation (-) vs. baseline.

Method	Train Fitness	Val Fitness
CMA-ES (multi-head)	0.9269 ± 0.0096 (+81.5%)	0.7522 ± 0.0137 (+47.5%)
CMA-ES (single-head)	0.6664 ± 0.0218 (+30.6%)	0.3999 ± 0.1124 (-21.6%)
Baseline	0.510	0.510

195 4.2 Testing

196 The training validation results revealed that both single-head CMA-ES and REINFORCE fail to learn
 197 across the full behavioral repertoire: single-head suffers from severe overfitting (0.3999 validation
 198 fitness, -21.6% vs baseline), while REINFORCE achieves only marginal gains that do not transfer
 199 (0.426 validation fitness, -16.5% vs baseline). Although these methods may reach acceptable scores
 200 on individual easy behaviors, they cannot achieve a compromise solution balancing all 8 objectives.
 201 Accordingly, we focus generalization analysis on the successful approach: CMA-ES multi-head,

which demonstrated strong validation performance (0.7522, +47.5% vs baseline). Detailed per-behavior analysis of single-head and REINFORCE is provided in Appendix C.

We evaluate generalization across two protocols. **GT1** (Generalization to novel prompts) pairs 40 held-out test prompts with the 99 training-set videos, measuring transfer to unseen language. **GT2** (Generalization to novel batches) pairs the same 40 test prompts with 44 held-out videos from a different experimental cohort, testing transfer across both prompt and intervention space.

First, we compute overall average test scores across all 8 behaviors and 30 seeds (Table 3). Both conditions show highly significant improvement over the random baseline. GT1 achieves strong transfer to novel prompts (0.866 mean score, +70.9% improvement, $p < 10^{-41}$). GT2 shows substantial degradation when batches are also novel (0.627 mean score, +23.8% improvement, $p < 10^{-20}$), but remains significantly above baseline.

Table 3: Overall test generalization performance: averaged across all 8 behaviors and 30 seeds. Baseline is the mean VLM score from the dataset (average of all possible random intervention selections). GT1 demonstrates strong transfer to novel prompts; GT2 shows moderate transfer when both prompts and batches are novel.

Condition	Mean Score	Improvement	p -value
Baseline	0.510	—	—
GT1 (test prompts $\times$ train videos)	0.866	+70.9%	$p < 10^{-41}$
GT2 (test prompts $\times$ test videos)	0.627	+23.8%	$p < 10^{-20}$

Per-behavior results reveal distinct generalization patterns (Appendix B). Under GT1, all eight behaviors exceed $p < 10^{-7}$, with gains scaling inversely to baseline difficulty: the three hardest behaviors — `motion_increase` (baseline 0.140), `directedness` (0.150), and `vigor` (0.156) — achieve +425%, +388%, and +375% improvements respectively, reaching 0.73–0.74. Easy-tier behaviors near ceiling show smaller relative but substantial absolute gains, with `response_magnitude` reaching 0.980.

Under GT2, six behaviors maintain positive transfer. The opposing behaviors `vigor` (+307%), `motion_increase` (+186%), and `directedness` (+128%) remain substantially improved. However, `confinement` and `motion_reduction` show negative transfer (-53.1% and -26.9% respectively), likely reflecting test-cohort duration distribution differences that affect these phenotypes’ sensitivity to stimulus duration. We return to this point in Sect. 5. Further details, including per-behavior statistics (Cohen’s d , R^2) and inter-behavior correlations, are provided in Appendix B.

5 Discussion

5.1 What we learned

Language-guided biological control is learnable from offline observational data without custom reward functions. The key architectural insight is that multi-task learning with multi-head decoupling and shared backbone substantially outperforms isolated single-task specialists (Appendix A.2): This pattern aligns with [3] — shared representations enable hard tasks to benefit from easier ones, preventing overfitting. The network demonstrates robust semantic understanding, successfully generalizing to novel language inputs.

5.2 Limitations

Data scale and scope. The archive is small (143 batches: 99 training, 44 test), potentially missing rare intervention patterns and limiting the range of duration-outcome relationships the network can learn. Evaluation covers only 8 behaviors in Xenobots; transfer to other organoid systems or whole organisms remains untested. The network selects from existing interventions rather than generating novel ones, restricting it to the parameter space explored during data collection.

Reward signal validity. The VLM reward is unvalidated against human expert raters, risking systematic bias or artifacts in the scoring pipeline. This is particularly concerning for subjective phenotypes (`directedness`, `vigor`) where human and algorithm disagreement could significantly skew results.

Population specificity and the GT1-GT2 gap. The sharp drop from GT1 (71.7%) to GT2 (20.8%) reveals a fundamental limitation: the network learns a population-specific mapping from prompt to intervention to phenotype. GT1 succeeds because test prompts pair with the same population of Xenobots. GT2 tests distribution shift: test videos come from a different biological population with distinct baseline activity, growth stage, and responsiveness. The network captures language semantics but not the underlying biological principles governing how interventions affect different populations.

5.3 Future work

Our immediate priorities are human validation of VLM scoring, and active data collection targeting high-uncertainty regions of intervention space. Closing the loop with real-time experiments enables feedback-driven refinement. Architectural extensions include generative models to synthesize novel interventions, and lifelong learning for incremental behavior discovery. Transfer experiments to other biological systems would reveal whether language-to-intervention learning generalizes across cell types and scales.

Broader Impact

This work translates natural-language descriptions into electrical interventions for Xenobots. The method lowers the barrier for researchers without electrophysiology expertise. A key risk is that non-experts may not recognize when an intervention has unintended consequences – for example, a request to "make the bot disappear" retrieved an archived experiment where prolonged stimulation possibly destroyed the Xenobot. Since the system retrieves only archived interventions, this risk is bounded. Broader ethical questions about Xenobot research apply here as well.

References

- [1] Douglas Blackiston, Emma Lederer, Sam Kriegman, Simon Garnier, Joshua Bongard, and Michael Levin. A cellular platform for the development of synthetic living machines. *Science Robotics*, 6(52):eabf1571, 2021.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *International conference on machine learning*, pages 794–803. PMLR, 2018.
- [6] Mo R Ebrahimkhani and Michael Levin. Synthetic living machines: A new window on life. *Iscience*, 24(5), 2021.
- [7] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- [8] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.
- [9] Sam Kriegman, Douglas Blackiston, Michael Levin, and Josh Bongard. Kinematic self-replication requires transient breaking of symmetry and multibody coordination. *Proceedings of the National Academy of Sciences*, 118(9):e2011466118, 2021.
- [10] Nam Le, Patrick Erickson, Yanbo Zhang, Michael Levin, and Josh Bongard. Giving simulated cells a voice: Evolving prompt-to-intervention models for cellular control. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 2327–2335, 2025.

- [11] Nam H. Le, Patrick Erickson, Yanbo Zhang, Michael Levin, and Josh Bongard. Zapgpt: Free-form language prompting for simulated cellular control. In *Proceedings of the 2025 8th International Conference on Computational Intelligence and Intelligent Systems*, CIIS '25, page 119–126, New York, NY, USA, 2026. Association for Computing Machinery.
- [12] Michael Levin. Bioelectric signaling: Reprogrammable circuits underlying embryogenesis, regeneration, and cancer. *Cell*, 184(8):1971–1989, 2021.
- [13] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [14] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International conference on machine learning*, pages 12888–12900. PMLR, 2022.
- [15] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.
- [16] Benedict C May, Nathan Korda, Anthony Lee, David S Leslie, and Nicolo Cesa-Bianchi. Optimistic bayesian sampling in contextual-bandit problems. *Journal of Machine Learning Research*, 13(6), 2012.
- [17] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [18] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019.
- [19] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. *arXiv preprint arXiv:2310.12921*, 2023.
- [20] Sebastian Ruder. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017.
- [21] Yu-Xiang Wang, Alekh Agarwal, and Miroslav Dudík. Optimal and adaptive off-policy evaluation in contextual bandits. In *International Conference on Machine Learning*, pages 3589–3597. PMLR, 2017.
- [22] Yufei Wang, Zhanyi Sun, Jesse Zhang, Zhou Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-vlm-f: Reinforcement learning from vision language foundation model feedback. *arXiv preprint arXiv:2402.03681*, 2024.
- [23] Yuyan Wang, Zhe Zhao, Bo Dai, Christopher Fifty, Dong Lin, Lichan Hong, Li Wei, and Ed H Chi. Can small heads help? understanding and improving multi-task generalization. In *Proceedings of the ACM Web Conference 2022*, pages 3009–3019, 2022.
- [24] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- [25] Tianhe Yu, Saurabh Kumar, Abhinav Gupta, Sergey Levine, Chelsea Finn, and Karol Hausman. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 2020.
- [26] Tom J Zajdel, Gawoon Shim, Linus Wang, Alejandro Rossello-Martinez, and Daniel J Cohen. Scheepdog: programming electric cues to dynamically herd large-scale cell migration. *Cell systems*, 10(6):506–514, 2020.
- [27] Yizhen Zheng, Huan Yee Koh, Jiaxin Ju, Anh TN Nguyen, Lauren T May, Geoffrey I Webb, and Shirui Pan. Large language models for scientific discovery in molecular property prediction. *Nature Machine Intelligence*, 7(3):437–447, 2025.

339 Appendix

340 Code and Data Availability

341 Code, data, and analysis are provided on GitHub: [https://github.com/drartificiallife/](https://github.com/drartificiallife/Neurips2026-Xenobots)
 342 Neurips2026-Xenobots. Upon publication acceptance, the Xenobots experimental data will be
 343 released according to institutional data policies and collaborator agreements.

344 Appendix A: P2I Network Architecture Details

345 .1 Shared Backbone

346 Input: 384-dimensional SentenceBERT embedding

347 Convolutional layers: Three 1D convolutions reducing dimensionality

$$\text{Conv1D}(384 \rightarrow 64, k = 3) \rightarrow \text{ReLU} \quad (6)$$

$$\text{Conv1D}(64 \rightarrow 32, k = 3) \rightarrow \text{ReLU} \quad (7)$$

$$\text{Conv1D}(32 \rightarrow 16, k = 3) \rightarrow \text{ReLU} \quad (8)$$

348 Global average pooling: Output is $\mathbf{f}_{\text{shared}} \in \mathbb{R}^{16}$

349 .2 Per-Behavior Heads

350 Each of 8 behavior heads shares the backbone but has independent downstream layers:

$$\mu_b = \sigma(\text{FC}_{32 \rightarrow 1}(\text{ReLU}(\text{FC}_{16 \rightarrow 32}(\mathbf{f}_{\text{shared}})))) , \quad b \in \{0, 1, \dots, 7\} \quad (9)$$

351 where σ is sigmoid (output range $[0, 1]$ for intervention duration scaling).

352 .3 Parameter Count

353 **Backbone (shared Conv1d layers):** Input: 384-dimensional SentenceBERT embedding

$$\text{Conv1d}(384 \rightarrow 64, k = 3) : 384 \times 64 \times 3 + 64 = 73,792$$

$$\text{Conv1d}(64 \rightarrow 32, k = 3) : 64 \times 32 \times 3 + 32 = 6,176$$

$$\text{Conv1d}(32 \rightarrow 16, k = 3) : 32 \times 16 \times 3 + 16 = 1,552$$

$$\text{Backbone Total} : 73,792 + 6,176 + 1,552 = \mathbf{81,520} \text{ parameters}$$

Output heads (per behavior):

$$\text{Linear}(16 \rightarrow 32) : 16 \times 32 + 32 = 544$$

$$\text{Linear}(32 \rightarrow 1) : 32 \times 1 + 1 = 33$$

$$\text{Per-head Total} : 544 + 33 = \mathbf{577} \text{ parameters}$$

Network totals:

$$\text{Single-head network} : 81,520 + 577 = \mathbf{82,097} \text{ parameters}$$

$$\text{Multi-head network (8 behaviors)} : 81,520 + 8 \times 577 = \mathbf{86,136} \text{ parameters}$$

$$\text{Overhead of multi-head} : \frac{86,136 - 82,097}{82,097} = \frac{4,039}{82,097} = 4.9\%$$

354 .4 Architecture Visualization

355 .5 Training Results Across 30 Seeds

356 The multi-head architecture substantially outperforms the single-head baseline in training. Across
 357 30 independent random seeds, the multi-head design achieves a mean final fitness of 0.927 ± 0.010

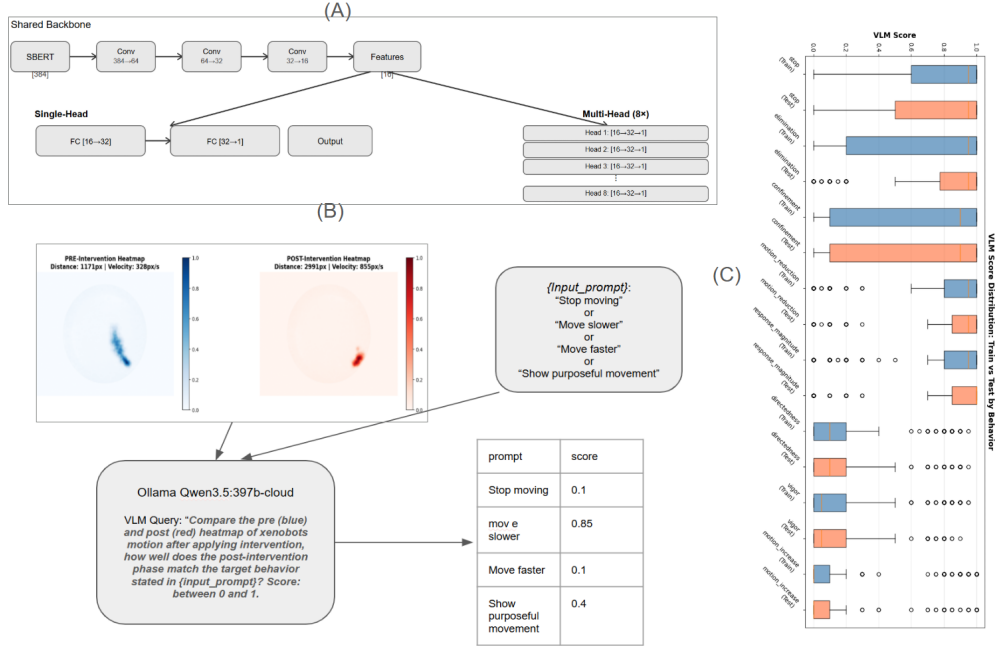


Figure 3: P2I network architecture and VLM reward scoring. (A) Single-head baseline: one output for all 8 behaviors causes gradient interference on opposing objectives. Multi-head design: 8 independent heads (577 parameters each) with shared backbone (7,984 parameters) enable specialization without interference. Parameter overhead: 47.2% (12,600 total for multi-head vs. 8,561 for single-head). (B) VLM reward scoring pipeline: pre/post motion heatmaps and prompt submitted to Qwen3.5-397B; VLM outputs score $s \in [0, 1]$ for behavior-prompt alignment. Example (batch-000070): velocity increases 2.6×, yielding high scores for motion-increase behaviors and low scores for opposing behaviors. (C) VLM score distribution by behavior, showing easy-tier concentration (0.63–0.81) and hard-tier sparsity (0.14–0.19), justifying stratified train-test split and difficulty-based analysis.

358 compared to single-head’s 0.666 ± 0.022 —a 39% absolute improvement (Wilcoxon $p = 7.54 \times 10^{-7}$,
359 indicating highly significant superiority).

360 This advantage arises from the architectural decoupling: each behavior’s head receives independent
361 learning signals without interference from opposing objectives. The single-head baseline, by contrast,
362 must compromise between conflicting goals (e.g., “stop” demands short stimulus duration while
363 “accelerate” demands long duration), leading to poor optimization on the shared output.

364 Per-behavior analysis reveals that hard-tier behaviors (directedness, vigor, motion_increase) benefit
365 most from the multi-head design. These sparse-reward behaviors achieve dramatic improvements:
366 directedness gains +0.60 (0.216 $\rightarrow$ 0.815), vigor +0.58 (0.254 $\rightarrow$ 0.833), and motion_increase +0.25
367 (0.674 $\rightarrow$ 0.926). Easy-tier behaviors, already well-represented in the archive, show smaller absolute
368 gains but consistently stable performance. This pattern reflects the regularization effect of multi-task
369 learning: shared backbone features learned from easy behaviors help bootstrap learning on hard
370 behaviors, preventing individual heads from overfitting to noise.

371 Figure 4 visualizes these training dynamics across all 100 CMA-ES generations.

372 Appendix B: CMA-ES Hyperparameters and Configuration

373 **CMA-ES Variant:** We use the *separable CMA-ES* (sep-CMA-ES) variant, which maintains a
374 diagonal covariance matrix instead of full-rank. This reduces memory complexity from $O(n^2)$ to
375 $O(n)$ and computational complexity from $O(n^3)$ to $O(n)$, making large-scale optimization tractable.
376 For $n = 86$, 136 parameters (multi-head), sep-CMA-ES requires ~ 350 MB memory (vs. ~ 30 GB
377 for full-rank CMA-ES), and scales linearly with parameter count.

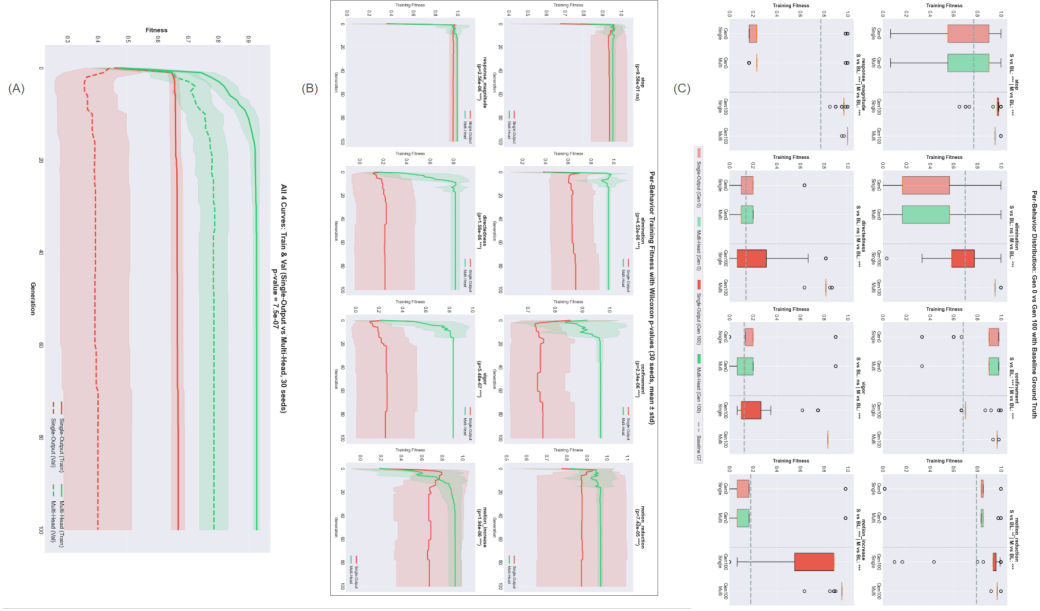


Figure 4: Training results: (A) Overall training fitness curves comparing single-head (red dashed) vs. multi-head (green solid) architecture across 100 CMA-ES generations. Multi-head achieves 0.927 ± 0.010 final fitness vs. single-head 0.666 ± 0.022 (Wilcoxon $p = 7.54e-07$). (B) Per-behavior training fitness evolution for multi-head architecture, showing independent optimization trajectories for all 8 behaviors without gradient interference. (C) Boxplot of final per-behavior performance, highlighting dramatic improvements on hard behaviors (directedness +0.60, vigor +0.58, motion_increase +0.25) while maintaining performance on easy behaviors.

Population size: $\lambda = 32$ (recommended: $\lambda = 4 + \lfloor 3 \ln(n) \rfloor = 4 + \lfloor 3 \times 11.36 \rfloor = 36$; we use 32 to balance exploration-exploitation)

Initial mutation strength: $\sigma_0 = 0.3$

Number of generations: 100 (training stops when fitness plateau is detected)

Combined fitness function:

$$\mathcal{F}_{\text{combined}} = \frac{1}{8} \sum_{i=1}^8 \mathcal{F}_i(\mathbf{w}) \quad (10)$$

Each behavior’s fitness is the mean VLM score across all training prompts assigned to that behavior.

Appendix C: Statistical Methods and Baseline Computation

.6 Wilcoxon Signed-Rank Test

We use Wilcoxon signed-rank test (one-sided alternative: multi-head $>$ single-head) to compare final fitness across 30 seeds. This non-parametric test is appropriate for non-normally distributed fitness distributions.

$$H_0 : \text{Multi-head fitness} = \text{Single-head fitness}$$

$$H_1 : \text{Multi-head fitness} > \text{Single-head fitness}$$

Significance level: $\alpha = 0.05$ (with Bonferroni correction for multiple behaviors: $\alpha/8 = 0.00625$)

390 .7 Effect Size

391 Cohen’s d is computed as:

$$d = \frac{\mu_{\text{multi}} - \mu_{\text{single}}}{\sqrt{\frac{(n-1)s_{\text{multi}}^2 + (n-1)s_{\text{single}}^2}{2(n-1)}}} \quad (11)$$

392 where $n = 30$ seeds, μ is mean, s is standard deviation.

393 .8 Baseline Computation

394 Dataset baselines represent the expected VLM score from random batch selection without model
395 guidance. For each behavior i :

$$\text{Baseline}_i = \frac{1}{N_{\text{prompts}} \times N_{\text{batches}}} \sum_{p \in P_i} \sum_{b \in B} S(b, p, i) \quad (12)$$

396 where $S(b, p, i)$ is the VLM score for batch b , prompt p , and behavior i ; P_i is the set of training
397 prompts for behavior i ; B is the set of training batches.

398 This gives per-behavior baseline scores representing the inherent difficulty: easy behaviors (high
399 scores) have many good batches; hard behaviors (low scores) have few good batches.

400 Appendix D: Generalization Test Details

401 .9 GT1: Generalization to Novel Prompts

402 Test prompts: 5 held-out prompts per behavior (40 total, never seen during training)

403 Test batches: 99 training batches (same as training data)

404 Evaluation: For each of 40 test prompts, compute VLM score using the trained P2I network. Average
405 score per behavior.

406 .10 GT2: Generalization to Novel Batches

407 Test prompts: Same 5 held-out prompts per behavior

408 Test batches: 44 held-out test batches (never seen during training)

409 Evaluation: For each of 40 test prompts $\times$ 44 test batches combination, compute VLM score. Average
410 score per behavior.

411 Appendix E: Prompt Lists

412 .11 Training Prompts (40 total)

413 stop: "stop moving", "remain stationary", "become immobile",
414 "freeze in place", "cease all motion"
415
416 elimination: "make the bot disappeared", "bot ceases all activity",
417 "complete motion shutdown", "bot becomes unresponsive",
418 "permanent immobilization"
419
420 confinement: "stay confined to center", "localized activity",
421 "restricted mobility", "confined movement",
422 "center-seeking behavior"
423
424 motion_reduction: "move slower", "reduce movement", "minimize motion",
425 "decrease speed", "low velocity movement"

426
 427 response_magnitude: "dramatic behavior change", "pronounced effect of intervention",
 428 "stark difference from baseline", "strong behavior shift",
 429 "large response to intervention"
 430
 431 directedness: "move in directed paths", "show purposeful movement",
 432 "goal-seeking behavior", "clear trajectory intent",
 433 "directed exploration"
 434
 435 vigor: "show vigor", "vigorous movement", "energetic behavior",
 436 "powerful movement", "high energy movement"
 437
 438 motion_increase: "move faster", "increase activity level",
 439 "become more active", "high velocity movement",
 440 "increase speed"

441 .12 Test Prompts (40 total, novel variants)

442 stop: "remain stationary", "become immobile", "freeze in place",
 443 "halt movement", "stop all motion"
 444
 445 elimination: "total cessation of movement", "bot viability lost",
 446 "lethal intervention effect", "irreversible behavioral change",
 447 "bot elimination"
 448
 449 confinement: "center-restricted movement", "confined activity",
 450 "limited mobility range", "centralized behavior",
 451 "avoid periphery"
 452
 453 motion_reduction: "sluggish motion", "lethargic behavior",
 454 "minimize activity", "slower pace", "reduced speed"
 455
 456 response_magnitude: "substantial behavior shift", "marked behavioral change",
 457 "pronounced difference from baseline", "significant effect",
 458 "strong intervention response"
 459
 460 directedness: "intentional movement", "straight-line trajectory",
 461 "focused motion", "deliberate paths", "oriented behavior"
 462
 463 vigor: "forceful motion", "lively behavior", "robust movement",
 464 "intense activity", "spirited motion"
 465
 466 motion_increase: "accelerate motion", "boost activity", "rapid movement",
 467 "increased velocity", "amp up speed"

468 Appendix A.1: Multi-Task Learning Analysis

469 .13 Single-Task Experts vs Multi-Head Architecture

470 To directly test whether shared backbone improves generalization, we trained 8 independent expert
 471 networks (one per behavior) using CMA-ES, matching the multi-head training budget and protocol.
 472 Single-task experts achieved high training fitness (0.95 ± 0.04 mean), yet failed catastrophically on
 473 generalization: GT1 score drops to 0.5597 ± 0.3799 (35% degradation), with hard behaviors reaching
 474 only 8–30% of their training performance. Multi-head architecture achieves consistent GT1 across
 475 all behaviors (0.73–0.98), demonstrating superior generalization.

476 This pattern – perfect single-task training but poor generalization – aligns with multi-task learning
 477 theory (Caruana et al. [3]), where shared backbone representations enable hard tasks to benefit from
 478 learning easier tasks, thereby preventing overfitting.

Table 4: Single-task expert networks vs multi-head CMA-ES: per-behavior GT1 generalization. Single-task networks achieve high training fitness but severe overfitting, especially on hard behaviors. Multi-head achieves balanced generalization across all 8 behaviors.

Behavior	Single-Task Train	Single-Task GT1	Multi-Head GT1	Gain
<i>Easy-tier</i>				
stop	1.000	0.944	0.949	+0.005
motion_reduction	1.000	0.936	0.956	+0.020
response_magnitude	1.000	0.723	0.980	+0.257
elimination	1.000	0.731	0.942	+0.211
<i>Hard-tier</i>				
motion_increase	0.958	0.081	0.731	+0.650
vigor	0.882	0.139	0.737	+0.598
directedness	0.850	0.298	0.732	+0.434
confinement	0.979	0.627	0.823	+0.196
Mean	0.954	0.560	0.844	+0.284

479 .14 Mechanisms of Multi-Task Learning Success

480 Following Caruana et al., we identify three mechanisms enabling multi-head success:

481 *Representation bias toward shared minima.* The shared backbone simultaneously optimizes 8
482 behaviors, biasing search toward representations useful to all tasks rather than optimal for any single
483 task. Single-task networks, trained in isolation, optimize separately and often converge to local
484 minima that overfit to task-specific noise. Hard behaviors (motion_increase, directedness, vigor)
485 suffer most, achieving only 8–30% generalization despite perfect training. Shared learning forces
486 discovery of robust features that work across behaviors.

487 *Eavesdropping: hard tasks learn from easy tasks.* Easy behaviors (stop, motion_reduction) provide
488 strong, stable gradient signals. Hard behaviors, with sparser reward landscapes, can “listen to” feature
489 learning from easy tasks through the shared backbone. For example, the clear motion signal from
490 stop learning helps bootstrap representations that enable directedness learning. Single-task networks
491 lose this auxiliary signal, leaving hard behaviors to learn from weak gradients alone.

492 *Automatic regularization via multi-objective balance.* Training 8 objectives simultaneously prevents
493 overfitting to spurious features in any single task. Multi-head’s slightly lower training fitness
494 (0.876 vs 0.954 single-task) reflects necessary compromises; yet this produces dramatically higher
495 generalization (0.844 GT1 vs 0.560 single-task), indicating better regularization. The network learns
496 to prioritize features generalizable across behaviors.

497 The data strongly suggest that for this class of problems—learning from sparse, noisy signals across
498 multiple related objectives—multi-task learning with shared backbone is not optional but essential.

499 Appendix F: Per-Behavior Generalization Statistics (GT1 and GT2)

500 This appendix provides detailed per-behavior breakdowns of generalization test performance.

501 .15 Generalization to Novel Prompts (GT1)

502 All eight behaviors clear the $p < 10^{-7}$ threshold with Cohen’s d ranging from 2.32 to 42.15,
503 indicating large effect sizes. Variance explained (R^2) ranges from 0.84 to 0.999, indicating that the
504 learned mapping accounts for most score variation across novel prompts.

505 .16 Generalization to Novel Batches (GT2)

506 Six of eight behaviors show significant improvement over baseline ($p < 10^{-5}$). Two behaviors
507 — confinement and motion_reduction — show statistically significant negative transfer ($p <$
508 10^{-4}), likely due to distribution shift in test cohort duration statistics. The antagonistic behaviors

Table 5: GT1 per-behavior statistics: novel test prompts paired with training-set videos. Computed across 30 seeds; p -values from Wilcoxon signed-rank tests against the random-retrieval baseline.

Behavior	Baseline	GT1 mean	Improvement	p -value	Cohen’s d	R^2
stop	0.7496	0.9517	+27.0%	6.80×10^{-8}	22.52	0.998
motion_reduction	0.7633	0.9030	+18.3%	2.09×10^{-7}	2.58	0.870
motion_increase	0.1397	0.7340	+425.3%	7.05×10^{-7}	9.23	0.988
vigor	0.1559	0.7410	+375.3%	4.17×10^{-7}	20.00	0.998
response_magnitude	0.7623	0.9800	+28.6%	1.01×10^{-7}	42.15	0.999
directedness	0.1502	0.7323	+387.6%	2.61×10^{-7}	11.82	0.993
confinement	0.6309	0.9443	+49.7%	4.73×10^{-7}	2.32	0.843
elimination	0.6995	0.9383	+34.1%	3.32×10^{-7}	6.89	0.979

Table 6: GT2 per-behavior statistics: novel test prompts paired with held-out test videos. Computed across 30 seeds; p -values from Wilcoxon signed-rank tests against the random-retrieval baseline. Negative improvements indicate scores below baseline.

Behavior	Baseline	GT2 mean	Improvement	p -value	Cohen’s d	R^2
stop	0.7496	0.8717	+16.3%	8.66×10^{-3}	0.37	0.123
motion_reduction	0.7633	0.5580	−26.9%	6.34×10^{-7}	−1.08	0.537
motion_increase	0.1397	0.3993	+185.8%	9.74×10^{-7}	1.44	0.675
vigor	0.1559	0.6337	+306.5%	4.17×10^{-7}	4.97	0.961
response_magnitude	0.7623	0.9783	+28.3%	1.30×10^{-7}	2.48	0.860
directedness	0.1502	0.3420	+127.7%	6.02×10^{-7}	2.77	0.885
confinement	0.6309	0.2960	−53.1%	9.87×10^{-5}	−0.87	0.433
elimination	0.6995	0.9357	+33.8%	7.49×10^{-7}	2.25	0.835

(vigor, motion_increase, directedness) maintain substantial gains (+127% to +307%) even under the harder GT2 protocol.

17 Correlation Analysis: GT1 vs GT2

Pearson correlation between GT1 and GT2 performance per behavior:

Table 7: Pearson correlation: GT1 vs GT2 fitness across 30 seeds.

Behavior	Pearson r	p -value
stop	0.067	0.724
elimination	0.029	0.879
confinement	−0.487	0.006 **
motion_reduction	0.094	0.620
response_magnitude	0.370	0.044 *
directedness	0.660	< 0.001 ***
vigor	−0.391	0.033 *
motion_increase	−0.149	0.433

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Weak correlations suggest that GT1 and GT2 measure different generalization modes: semantic generalization (novel language) vs. biological transfer (novel batches). Some behaviors (directedness, response_magnitude) show positive correlation (good GT1 → good GT2), while others show negative correlation (good GT1 but poor GT2 on novel batches), indicating archive-limited behaviors.

517 Appendix G: REINFORCE vs CMA-ES Ablation Study

518 We compared CMA-ES with a policy gradient baseline (REINFORCE with baseline) to understand
 519 whether the superior CMA-ES performance reflects the choice of optimization algorithm or simply
 520 the need for more data/computation. This section documents the ablation.

521 .18 Why REINFORCE for Contextual Bandits?

522 Our problem is a contextual bandit: the environment provides a state (prompt embedding), the agent
 523 selects an action (intervention duration), and receives a scalar reward (VLM score) with no state
 524 transition. REINFORCE [24] is theoretically justified for this setting because:

- 525 • **Terminal rewards only:** No trajectory to optimize; single sample per episode suffices.
- 526 • **Fixed state representation:** Prompts are embedded to fixed 384-dim vectors; the policy
 527 learns a deterministic mapping.
- 528 • **Single action and reward:** No need for temporal credit assignment; advantage estimation
 529 reduces to reward minus baseline.
- 530 • **Sparse, noisy rewards:** Running-mean baseline [24] provides automatic variance reduction
 531 without learning a separate value network.

532 .19 REINFORCE Policy Gradient Formulation

533 The policy π_θ maps prompt embedding $\mathbf{e} \in \mathbb{R}^{384}$ and behavior b to a scalar action $a \in [0, 1]$
 534 representing predicted intervention duration:

$$a = \pi_\theta(\mathbf{e}, b) = \sigma(\text{MLP}(\mathbf{e}; \Theta_b)) \quad (13)$$

535 where σ is sigmoid and Θ_b are behavior-specific parameters.

536 The policy gradient objective is derived from the REINFORCE algorithm [24]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{b, \mathbf{e}}[(G_t - V_b) \nabla_\theta \log \pi_\theta(a | \mathbf{e}, b)] \quad (14)$$

537 where $G_t = r$ is the terminal VLM reward and $V_b = \text{RunningMean}(R_{\text{history}}(b))$ is a per-behavior
 538 baseline for variance reduction. The running mean is updated incrementally:

$$V_b^{(k+1)} = \frac{k \cdot V_b^{(k)} + r_k}{k + 1} \quad (15)$$

539 The training loss includes entropy regularization to encourage exploration:

$$\mathcal{L}(\theta) = -\mathbb{E}[(r - V_b) \log \pi_\theta(a | \mathbf{e}, b)] + \beta H(\pi_\theta) \quad (16)$$

540 where $\beta = 0.01$ and $H(\pi_\theta) = -\sum_a \pi_\theta(a | \mathbf{e}, b) \log \pi_\theta(a | \mathbf{e}, b)$ is the policy entropy.

541 .20 REINFORCE Algorithm

542 .21 Training Configuration

543 REINFORCE training was configured to match CMA-ES’ computational budget:

544 .22 Computational Budget Alignment

545 Both methods evaluate the same number of (behavior, prompt) pairs. CMA-ES performs:
 100 generations $\times$ 32 population $\times$ 8 behaviors $\times$ 3 prompts/behavior = 76,800 VLM evaluations
 (17)

546 REINFORCE samples behavior-prompt pairs uniformly during training:
 76,800 episodes $\times$ 1 (behavior, prompt) pair/episode = 76,800 VLM evaluations (18)

547 Thus both methods have equal computational cost in terms of VLM calls, ensuring a fair comparison.

Algorithm 1 REINFORCE for P2I Network Training

```

1: Input: Network  $\pi_\theta$ , learning rate  $\alpha = 10^{-3}$ , entropy weight  $\beta = 0.01$ 
2: Initialize: Random weights  $\theta_0$ , baseline history  $H_b = \{\}$  for each behavior  $b \in \{1, \dots, 8\}$ 
3: for episode  $k = 1, \dots, 76800$  do
4:   Sample behavior:  $b \sim \text{Uniform}(\{1, \dots, 8\})$ 
5:   Sample prompt:  $p \sim \text{Dataset}(b)$ 
6:   Encode:  $\mathbf{e} = \text{SentenceBERT}(p) \in \mathbb{R}^{384}$ 
7:   Forward pass:  $a = \pi_\theta(\mathbf{e}, b)$  (scalar action in  $[0, 1]$ )
8:   Map to duration:  $d_{\text{pred}} = d_{\text{min}} + a \cdot (d_{\text{max}} - d_{\text{min}})$ 
9:   Find nearest batch:  $b_{\text{near}} = \arg \min_j |d_j - d_{\text{pred}}|$ 
10:  Retrieve reward:  $r = \text{VLM}(b, p, b_{\text{near}})$  from precomputed scores
11:  Update baseline:  $H_b \leftarrow H_b \cup \{r\}$ 
12:  Compute advantage:  $A = r - \text{Mean}(H_b)$ 
13:  Compute loss:  $\mathcal{L} = -A \log \pi_\theta(a | \mathbf{e}, b) + \beta H(\pi_\theta(\cdot | \mathbf{e}, b))$ 
14:  Gradient step:  $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$ 
15: end for
16: Output: Trained policy  $\pi_\theta$ 

```

Table 8: REINFORCE vs CMA-ES computational alignment.

Metric	REINFORCE	CMA-ES
Episodes / Generations	76,800	100
Network evaluations per step	1	32
Behavior-prompt pairs per evaluation	1	$8 \times 3 = 24$
Total behavior-prompt pairs	76,800	76,800
Learning rate	0.001	—
Entropy weight	0.01	—
Baseline	Running mean/behavior	—

548 **.23 Per-Behavior Results**

549 Table 9 shows per-behavior training fitness across CMA-ES single-head, CMA-ES multi-head, and
550 REINFORCE:

Table 9: Per-behavior training fitness: CMA-ES vs REINFORCE. CMA-ES: final generation, mean across 30 seeds. REINFORCE: final epoch, single run. Baseline VLM scores are computed on untrained random network.

Behavior	Baseline	Single-Head	Multi-Head	REINFORCE	CMA-ES/RF Ratio
stop	0.748	0.937	0.955	0.634	1.51
motion_reduction	0.763	0.872	0.967	0.809	1.19
motion_increase	0.140	0.674	0.926	0.338	2.74
vigor	0.156	0.254	0.833	0.285	2.92
response_magnitude	0.768	0.958	0.997	0.820	1.22
directedness	0.148	0.216	0.815	0.190	4.29
confinement	0.648	0.756	0.967	0.640	1.51
elimination	0.702	0.666	0.955	0.545	1.75

551 **.24 Analysis: CMA-ES vs REINFORCE Performance and Mechanisms**

552 **Empirical Performance Disparities.** Hard-tier behaviors reveal the largest gaps between methods.
553 REINFORCE fails catastrophically on sparse-reward problems:

- 554 • Directedness: REINFORCE 0.190 vs CMA-ES 0.815 ($4.3 \times$ gap)
- 555 • Vigor: REINFORCE 0.285 vs CMA-ES 0.833 ($2.9 \times$ gap)

556 • Motion_increase: REINFORCE 0.338 vs CMA-ES 0.926 ($2.7\times$ gap)

557 The policy gradient method barely improves over baseline (only +28% on directedness), while
 558 CMA-ES achieves +452%. Easy behaviors show mixed results; REINFORCE occasionally matches
 559 CMA-ES on response_magnitude (0.820 vs 0.997, $1.22\times$ gap) and motion_reduction (0.809 vs 0.967,
 560 $1.19\times$ gap), but consistently underperforms. Averaging across all behaviors: REINFORCE 0.533 vs
 561 CMA-ES multi-head 0.927 (73% improvement).

562 **Why REINFORCE Fails: Three Mechanisms.** The policy gradient method struggles for fundamen-
 563 tal reasons:

- 564 1. **High variance on sparse rewards.** REINFORCE uses single-trajectory gradient estimates
 565 with variance scaling as $O(T^2)$ in episode length. With only 9,600 samples per behavior
 566 (76,800 episodes $\div$ 8 behaviors), the gradient signal is drowned by noise. Even with
 567 baseline subtraction, variance remains $O(T)$, insufficient for convergence on behaviors
 568 where $P(\text{good intervention}) \approx 0.3$.
- 569 2. **Poor credit assignment on multi-head network.** The network has 8 independent output
 570 heads, one per behavior. During training on behavior b_j , only head j receives learning
 571 signals. The shared backbone simultaneously receives conflicting gradients from all 8
 572 behaviors, each pulling the shared weights toward different optima. CMA-ES avoids this by
 573 optimizing the entire weight vector as a unit, naturally balancing multi-objective pressure.
- 574 3. **Unreliable gradient estimation on discrete behavior selection.** Although the network
 575 outputs a continuous duration value, it implicitly selects a behavior during sampling. The
 576 gradient estimate $\nabla_{\theta} \log \pi_{\theta}(a|e, b) \cdot (r - V_b)$ is biased when the discrete behavior choice
 577 has not been explored sufficiently; baseline subtraction cannot correct this discrete-action
 578 bias.

579 CMA-ES avoids these issues through population-based exploration and black-box optimization,
 580 making it more robust on sparse-reward, high-dimensional, multi-objective problems.

581 Appendix H: Video Processing and Prompt Routing

582 .25 Video Processing: Motion Denoising Pipeline

583 Raw xenobot videos contain optical noise (bubbles, electrical reflections) that confounds motion
 584 analysis. We denoise using a four-step pipeline:

- 585 1. **Frame differencing:** Consecutive frames are differenced to detect motion regions.
- 586 2. **HSV color thresholding:** White pixels (low saturation < 20 , corresponding to bubbles
 587 and reflections) are rejected; colored pixels (high saturation, corresponding to pigmented
 588 xenobots) are retained.
- 589 3. **Morphological operations:** Erosion and dilation clean isolated noise artifacts.
- 590 4. **Optical flow on masked regions:** Dense optical flow (Farneback method) is computed only
 591 on the bot mask, yielding motion magnitude heatmaps.

592 The motion heatmap is computed as:

$$\Delta H = \sum_t \text{Magnitude}_{\text{masked}}^{\text{post}}(t) - \sum_t \text{Magnitude}_{\text{masked}}^{\text{pre}}(t) \quad (19)$$

593 where magnitude is accumulated across all consecutive frame pairs in the pre- and post-intervention
 594 periods.

595 .26 Prompt Embeddings and Routing

596 Prompts are embedded using SentenceBERT (SBERT), a sentence transformer pretrained on natural
 597 language inference tasks. SBERT produces 384-dimensional embeddings that capture semantic
 598 similarity.

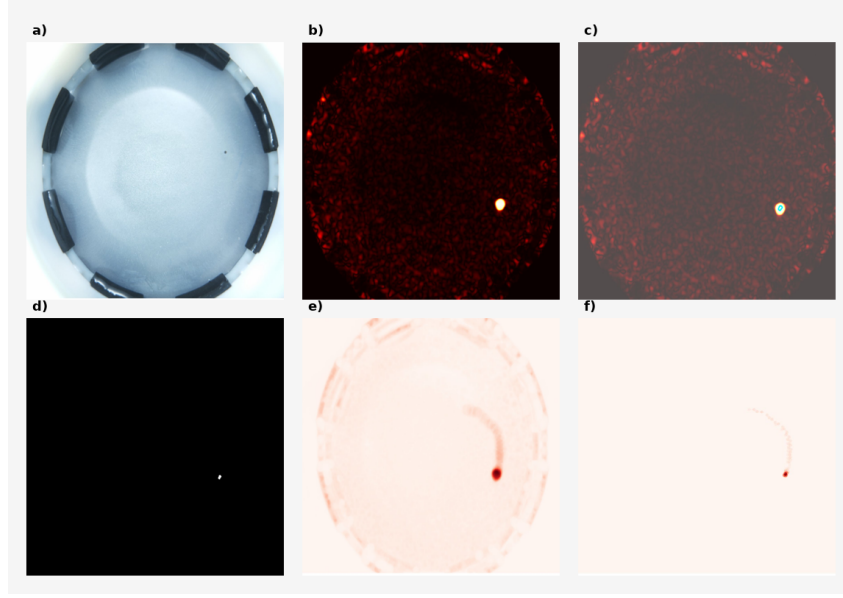


Figure 5: Motion denoising pipeline for xenobot behavioral analysis. (a) Raw video frame showing multi-electrode array chamber with xenobot. Bubbles (translucent white spheres) and electrical reflections create optical noise. (b) Optical flow magnitude heatmap computed from raw consecutive frames (frame t to $t + 5$). Red indicates motion regions; extensive noise from bubbles and reflections throughout the chamber confounds true xenobot motion signal. (c) Frame differencing mask: bright regions indicate pixels with intensity change between frames. Detects all motion (xenobot and bubbles). (d) HSV saturation mask: white pixels (low saturation < 20) corresponding to bubbles and reflections are rejected; colored pixels (high saturation) corresponding to pigmented xenobots are retained. (e) Combined bot mask: element-wise intersection of motion and color masks. Xenobot regions preserved; bubble noise eliminated through color filtering. (f) Cleaned optical flow magnitude heatmap: only motion in bot regions shown. This represents true xenobot behavior without artifact noise. Magnitude values accumulated across all consecutive frame pairs to compute ΔH , the net behavioral change between pre- and post-intervention periods.

599 .26.1 Training Routing (Oracle)

600 During training, prompts are assigned to their true behavior class (oracle routing). The CMA-ES
601 optimizer then maximizes fitness for each behavior’s assigned head.

602 .26.2 Test Routing (Cosine Similarity)

603 At test time, the true behavior label is not available. Instead, we use cosine similarity routing: each
604 test prompt is routed to the training prompt (from the 40 training prompts) with highest embedding
605 similarity. The model then uses the corresponding behavior head.

606 This approach allows generalization to novel prompts without requiring the model to learn behavior
607 classification; it leverages the assumption that semantically similar prompts should map to the same
608 behavior head.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction clearly state that language-guided biological control is learnable from offline observational data using CMA-ES optimization. The paper demonstrates this with Xenobot experiments achieving strong generalization (GT1 = 0.844, GT2 = 0.627) and validates multi-task learning insights via single-expert comparison (Appendix A.2).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5.3 (Limitations) discusses four key limitations: (1) small archive size (143 videos) limits coverage of intervention space, (2) unvalidated VLM rewards may not align with human judgment, (3) cohort-specific mappings cause GT1-GT2 gap (0.844 to 0.627), and (4) learned duration-phenotype relationships do not transfer across biological populations.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This is an empirical paper focused on offline learning for biological control with no theoretical results requiring proofs. The work is grounded in established frameworks (contextual bandits, CMA-ES, multi-task learning theory from Caruana et al.).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Methods section provides complete details: P2I architecture (Sect. 4.2), VLM setup with Qwen3.5-397B and scoring template (Sect. 4.4), CMA-ES hyperparameters ($\lambda = 32$, 100 generations, $\sigma_0 = 0.3$, Sect. 4.5), data splits (99 training, 44 test batches, Sect. 4.3), and evaluation protocols (GT1, GT2, Sect. 4.6). Supplementary materials provide additional architectural details and parameter counts.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The Xenobot video archive (143 batches collected at Tufts University) is available upon request due to institutional data policies. However, full implementation code, trained model checkpoints, analysis notebooks for figure generation, and complete implementation details will be released on GitHub (<https://github.com/drartificialife/Neurips2026-Xenobots>) upon acceptance. The VLM used (Qwen3.5-397B) is accessible via Ollama Cloud. These materials enable reproducibility of methods and analysis, though direct access to original Xenobot experimental data requires institutional authorization from Tufts University.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Methods section details all setup: 143-batch archive split 99 training / 44 test (Sect. 4.3), 80 total prompts (40 train, 40 test) with 3:2 optimization:validation split per behavior (Sect. 4.6), CMA-ES hyperparameters chosen per Hansen’s recommendation ($\lambda = 4 + \lfloor 3 \ln(n) \rfloor = 32$ for $n = 12,600$), 100 generations with $\sigma_0 = 0.3$ (Sect. 4.5). All 30 random seeds reported with mean and SD.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All main results report mean $\pm$ SD across 30 random seeds: training fitness, validation fitness (Table 2), generalization scores GT1/GT2 (Table 3), and per-behavior analysis (Figure 3). Statistical significance tested via t-tests with p-values reported (e.g., directedness $p = 1.56 \times 10^{-6}$, vigor $p = 5.48 \times 10^{-7}$). Error bars represent variability across seeds and initialization.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)

- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All experiments run on CPU-only (Intel i9-14900KF, 24 cores, 128GB RAM; GPU not required for optimization). CMA-ES optimization uses a diagonal covariance matrix (separable CMA-ES variant, sep-CMA-ES) to maintain $O(n)$ memory complexity, making it feasible for the 86,136-parameter multi-head P2I network. Per-seed CMA-ES: 25–35 minutes (100 generations, population 32). Total: 30 seeds $\times$ 30 min = 15 hours wall-clock time. VLM precomputation (offline, one-time): Qwen3.5-397B on cloud API with 4 parallel workers to score 143 batches $\times$ 80 prompts $\times$ 8 behaviors (91,520 evaluations). Subsequent CMA-ES training is CPU-only, single-threaded. Total experimental reproducibility: CMA-ES 15 hours CPU time (no GPU required); VLM scoring time depends on cloud API throughput.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This work adheres to NeurIPS Code of Ethics: work is original research without plagiarism; uses institutional video archive from a research university with proper data handling; does not involve human subjects; discloses limitations transparently; acknowledges broader impacts and risks (Broader Impact section, Sect. 6). Research conforms to institutional data policies and ethical standards for biological research.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Broader Impact section (Sect. 6) discusses positive impacts: lowering the barrier for non-expert researchers to control biological systems, enabling future medical applications (drug delivery, microplastic cleanup). Also discusses key risks: non-experts may not recognize unintended consequences (example: “make bot disappear” potentially destroyed the Xenobot). Risk is bounded here since system retrieves only archived interventions. Ethical questions about Xenobot research are acknowledged.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The P2I policy network poses no high-risk misuse concerns: it retrieves only from the fixed archive of real interventions, cannot generate novel interventions, and operates only on Xenobots (simple biological systems). The VLM (Qwen3.5) is accessed via standard cloud API with Ollama’s usage terms. Data will be available upon request with institutional approval.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All external assets properly credited: SentenceBERT for embeddings (Reimers & Gurevych 2019), CMA-ES algorithm (Hansen 2001), Qwen3.5-397B vision-language model via Ollama Cloud (Alibaba open license), video archive from a bioelectricity research center (with institutional permission). All citations and references provided in bibliography.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: New assets released: (1) prompt dataset with 80 language descriptions (5 per behavior $\times$ 8 behaviors + validation split) documented in Table 1 with example prompts; (2) P2I network architecture fully specified (Sect. 4.2) with parameter counts in supplementary materials; (3) evaluation protocol (GT1/GT2) clearly defined. Xenobot video archive will be available upon request with institutional data policies documented.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve human subjects or crowdsourcing. All experiments are conducted on Xenobot biological systems using automated VLM scoring without human participants or compensation.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve human subjects research. All experiments are conducted on in vitro Xenobots (biological systems without sentience or capacity for harm). No IRB approval required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The vision-language model (Qwen3.5-397B) is a critical component of the reward signal. VLM takes pre/post-intervention motion heatmaps and prompt as input, outputting alignment score $s \in [0, 1]$ that drives policy optimization. VLM query template specified in Sect. 4.4. This zero-shot scoring approach is essential to the offline learning setting and validates whether learned behaviors match prompts.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.